# Table 1. Experimental Evaluation of CKKS Matching Optimization

Deterministic work reduction, environment-dependent runtime, held-out matching quality, and kernel-level cost profile

| Category / Dimension | Metric / Parameter | Experimental Value / Result |
|---|---|---|
| **I. SYSTEM CONFIGURATION AND REPRODUCIBILITY** | | |
| Workload | CKKS encrypted matching; Round 2 tiled batch-matching kernel | Online Round 2 kernel only; not end-to-end protocol runtime |
| Candidate matrix / feature dimension | Baseline: c = 483; optimized: c = 400; d = 50 | Fixed, public-shape truncation |
| Tiles / selector multiplications | 25 → 20 tiles; 62,500 → 50,000 multiplications | –20% in both counts |
| Output consistency | First 400 columns; 80,000 slots compared | Zero slot-wise differences; max\|diff\| = 0 |
| Candidate coverage | Position coverage = 0.995 | 0.5% tail candidates discarded; approximation trade-off |
| **II. INTERLEAVED A/B/C BENCHMARK (PYTHON 3.12)** | | |
| A — Original kernel, c = 483 | 25 tiles; 62,500 multiplications | 73.57 s; 1.000× reference |
| B — Kernel v2, c = 483 | 25 tiles; 62,500 multiplications | 66.54 s; 1.106× |
| C — Kernel v2, c = 400 | 20 tiles; 50,000 multiplications | 53.26 s; 1.381× |
| Cross-environment replication | Python 3.12: v2 only 1.106×; combined 1.381× | Python 3.14: v2 only 1.000×; combined 1.230× |
| Interpretation | plain_tensor benefit depends on the TenSEAL wheel; nearly zero on cp314 | Report 1.23×–1.38× as a range; emphasize deterministic operation-count reduction |
| **III. MATCHING QUALITY (INDEPENDENT DEV/TEST SETS, 600 EACH)** | | |
| Threshold τ | Baseline: 0.90; current: 0.84 | Operating-point change; does not reduce computation |
| Test recall | 0.805 → 0.883 | +7.8 percentage points |
| Query-level FPR | 0/600 → 1/600 | 0.167%; current 95% upper bound ≈ 0.94% |
| Uncertainty / split isolation | Dev/test are separate from the tuning set | Recall 95% CI: [0.855, 0.907] |
| **IV. KERNEL CHANGES AND MEASURED COST PROFILE** | | |
| Plaintext construction | List input → ts.plain_tensor(..., dtype='float') | Avoids repeated list-to-plaintext conversion |
| Ciphertext accumulation | acc_f = acc_f + term → acc_f += term | Reduces repeated ciphertext allocation; same mathematical structure |
| Per-tile timing | ct-pt multiplication: 2.207 s / 88.1%; plaintext construction: 0.161 s / 6.4% | ct-ct addition: 0.073 s / 2.9%; dot products: 0.065 s / 2.6% |
| Structural bottleneck | 2,500 ct-pt multiplications per tile; slot utilization = 97.7% | Limited headroom for reduction-tree restructuring |

Notes: Runtime varies with environment, TenSEAL wheel implementation, and background load. The 1.381× speedup and +7.8-point recall change must not be multiplied: the former is a kernel timing result, while the latter comes from moving the decision threshold.